

Ονοματεπώνυμο: Αθανάσιος Εξάρχου

AM: it2022134

Εισαγωγή



Ευκλείδης (350 π.Χ. – 270 π.Χ.)



Blaise Pascal (1623–1662)

Το θεώρημα του διωνύμου δίνει τους συντελεστές του αναπτύγματος δυνάμεων διωνυμικών εκφράσεων και βρίσκει εφαρμογές στην οικονομία (οικονομικές προβλέψεις), στη μηχανική μάθηση (machine learning), στις χρηματιστηριακές εφαρμογές, στην αρχιτεκτονική, στη θεωρία πιθανοτήτων και τη στατιστική (διωνυμική κατανομή).

Ιστορικά, ο Ευκλείδης περί το 300 π.Χ. ήταν ο πρώτος επιστήμονας ο οποίος εφάρμοσε το θεώρημα του διωνύμου για $n=2$, και στη σύγχρονή του μορφή εκφράστηκε από τον Pascal το 1665 (πηγή: <https://mathworld.wolfram.com/BinomialTheorem.html>)

Οι διωνυμικοί συντελεστές είναι θετικοί ή μηδενικοί αριθμοί. Η σημερινή εργασία έχει ως στόχο τον υπολογισμό των συντελεστών του αναπτύγματος δυνάμεων του θεωρήματος του διωνύμου σε δύο βήματα.

Το πρώτο βήμα βαθμολογείται 1.0 βαθμό και το δεύτερο βήμα με επιπρόσθετο +0.5 βαθμό.

Χρησιμοποιώντας συμβολική γλώσσα MIPS και την αναδρομική διαδικασία υπολογισμού του παραγοντικού, υπολογίστε τον συντελεστή:

$$2C_5 = \binom{5}{2} = \frac{5!}{2!3!} = \frac{120}{6 \cdot 2} = 10$$

- Α. Υπολογίστε το $\binom{5}{2}$ χρησιμοποιώντας μια αναδρομική διαδικασία υπολογισμού παραγοντικού, και εφαρμόζοντας τις εντολές **mul**, **div** που βρίσκονται στο παράρτημα.
- Β. Εκτυπώστε το αποτέλεσμα στην οθόνη (MARS) χρησιμοποιώντας την εντολή syscall.
- Γ. Ενσωματώστε τον κώδικά σας με τα απαραίτητα σχόλια στο έγγραφο αυτό.
- Δ. Ενσωματώστε ένα snapshot των αποτελεσμάτων στο MARS στο έγγραφο αυτό.

Κώδικας - Βήμα 1

```
.text
main:
addiu $a0, $zero, 5 #αρχικοποιώ το όρισμα με 5
jal myfactorial
move $s0,$v0 #μεταφέρω το αποτέλεσμα σε μια μεταβλητή

li $a0, 2 #θέτω τη τιμή του ορίσματος ως 2
jal myfactorial
move $s1,$v0 #μεταφέρω το αποτέλεσμα σε μια μεταβλητή

li $a0, 3 #θέτω τη τιμή του ορίσματος ως 3
jal myfactorial
move $s2,$v0 #μεταφέρω το αποτέλεσμα σε μια μεταβλητή

mul $s3, $s1, $s2 #πολ/ζω τον παρονομαστή
div $s4, $s0, $s3 #διαίρω τον αριθμητή με τον παρονομαστή

move $a0, $s4 #μεταφέρω το αποτέλεσμα της διαίρεσης στον καταχωρητή $a0 για
εκτύπωση
li $v0, 1 #κωδικός για εκτύπωση
syscall #καλώ το σύστημα και εκτυπώνω

exit_module: li $v0, 10 #10= system code for exit
syscall #καλώ το σύστημα και βγαίνω

myfactorial: addi $sp, $sp, -8 #προσαρμόζω τη στοίβα για δύο στοιχεία(λαμβάνω
τους κατάλληλους πόρους)
sw $ra, 4($sp) #αποθηκεύω τη διεύθυνση επιστροφής
sw $a0, 0($sp) #αποθηκεύω το όρισμα
slt $t0, $a0, 1 #ελέγχω κατά πόσο έχω φθάσει σε φύλλο
beq $t0, $zero, L1
li $v0, 1 #εάν έχω φθάσει σε φύλλο, θέτω την τιμή 1 και επιστρέφω
addi $sp, $sp, 8 #αποδεσμεύω τη μνήμη
jr $ra #επιστρέφω στον καλούντα

L1: addi $a0, $a0, -1 #εκτελείται εάν δεν έχω φθάσει σε φύλλο

jal myfactorial #καλώ αναδρομικά

lw $a0, 0($sp) # μετά την επιστροφή από την αναδρομή, ανακτώ το όρισμα
lw $ra, 4($sp) # μετά την επιστροφή από την αναδρομή ανακτώ τη διεύθυνση
επιστροφής
addi $sp, $sp, 8 #αποδεσμεύω τη μνήμη
mul $v0, $a0, $v0 #πολ/ζω για να λάβω το αποτέλεσμα

jr $ra #γυρνώ ένα βήμα πίσω στην ιεραρχία της αναδρομής
```

Ακολουθούμενη λογική - Βήμα 1

Βρίσκεται στα σχόλια

Πίνακας ρόλου καταχωρητών – Βήμα 1

\$v0	Carries the results and executes certain codes for printing and exiting the program via syscall
\$ra	Contains the return address and enables us to go back to right-after the 'jal myfactorial' by using jr(jump on register)
\$sp	\$sp(stack pointer) is used to allocate and deallocate memory according to our needs in order to be able to have enough memory space to store and load words
\$t0	Temporary register used in order to check whether we've reached a leaf
\$s4	Contains the result of the division of the factorial of 5 and the multiplication of the factorials of 2 and 3

υλοποιώντας την με ένα βρόχο στο κυρίως πρόγραμμα, ώστε το k να λαμβάνει τις τιμές: $k=0, \dots, 5$ και εφαρμόζοντας το γενικό τύπο:

$$\binom{5}{k} = \frac{5!}{k!(5-k)!}$$

Β. Εκτυπώστε σε κάθε επανάληψη του βρόχου το αντίστοιχο αποτέλεσμα $\binom{5}{k}$ στην οθόνη (MARS) χρησιμοποιώντας την εντολή `syscall` εντός του βρόχου.

Γ. Ενσωματώστε τον κώδικά σας στο έγγραφο αυτό.

Δ. Ενσωματώστε ένα snapshot των αποτελεσμάτων στο MARS στο έγγραφο αυτό.

Ε. Συμπληρώστε το ανάπτυγμα του δυωνύμου με βάση τους συντελεστές που έχετε υπολογίσει:

$$(x+y)^5 = \sum_{k=0}^5 \binom{5}{k} x^{5-k} y^k = \binom{5}{0} x^{5-0} y^0 + \binom{5}{1} x^{5-1} y^1 + \binom{5}{2} x^{5-2} y^2 + \binom{5}{3} x^{5-3} y^3 + \binom{5}{4} x^{5-4} y^4 + \binom{5}{5} x^{5-5} y^5$$

Απάντηση:

$$1x^5 + 5x^4y + 10x^3y^2 + 10x^2y^3 + 5x^1y^4 + 1y^5$$

Κώδικας - Βήμα 2

```
.text
addiu $t1, $zero, 0
main:
beq $t1, 6, exit_module #βγαίνει απ'το πρόγραμμα όταν το k γίνει 1
addiu $a0, $zero, 5 #αρχικοποιώ το όρισμα με 5
jal myfactorial
sub $t2, $a0, $t1
move $s0,$v0 #μεταφέρω το αποτέλεσμα σε έναν αποθηκευμένου-τύπου
καταχωρητή

move $a0, $t1 #θέτω τη τιμή του ορίσματος
jal myfactorial
move $s1,$v0 #μεταφέρω το αποτέλεσμα σε έναν αποθηκευμένου-τύπου
καταχωρητή

li $a0, 0
add $a0, $t1, $t2 #θέτω τη τιμή του ορίσματος
jal myfactorial
move $s2,$v0 #μεταφέρω το αποτέλεσμα σε έναν αποθηκευμένου-τύπου
καταχωρητή

mul $s3, $s1, $s2 #πολ/ζω τον παρονομαστή
div $s4, $s0, $s3 #διαιρώ τον αριθμητή με τον παρονομαστή

move $a0, $s4 #μεταφέρω το αποτέλεσμα της διαίρεσης στον καταχωρητή $a0 για
εκτύπωση
```

```

li $v0, 1 #κωδικός για εκτύπωση
syscall #καλώ το σύστημα και εκτυπώνω
li $a0, 44
li $v0, 11 #κωδικός για εκτύπωση του ", "
syscall #καλώ το σύστημα και εκτυπώνω
addi $t1, $t1, 1 #αυξάνω τον δείκτη k κατά 1
j main

exit_module: li $v0, 10 #10= system code for exit
syscall #καλώ το σύστημα και βγαίνω

myfactorial: addi $sp, $sp, -8 #προσαρμόζω τη στοίβα για δύο στοιχεία(λαμβάνω
τους κατάλληλους πόρους)
sw $ra, 4($sp) #αποθηκεύω τη διεύθυνση επιστροφής
sw $a0, 0($sp) #αποθηκεύω το όρισμα
slti $t0, $a0, 1 #ελέγχω κατά πόσο έχω φθάσει σε φύλλο
beq $t0, $zero, L1
li $v0, 1 #εάν έχω φθάσει σε φύλλο, θέτω την τιμή 1 και επιστρέφω
addi $sp, $sp, 8 #αποδεσμεύω τη μνήμη
jr $ra #επιστρέφω στον καλούντα

L1: addi $a0, $a0, -1 #εκτελείται εάν δεν έχω φθάσει σε φύλλο
jal myfactorial #καλώ αναδρομικά
lw $a0, 0($sp) # μετά την επιστροφή από την αναδρομή, ανακτώ το όρισμα
lw $ra, 4($sp) # μετά την επιστροφή από την αναδρομή ανακτώ τη διεύθυνση
επιστροφής
addi $sp, $sp, 8 #αποδεσμεύω τη μνήμη
mul $v0, $a0, $v0 #πολ/ζω για να λάβω το αποτέλεσμα
jr $ra #γυρνώ ένα βήμα πίσω στην ιεραρχία της αναδρομής

```

Ακολουθούμενη λογική - Βήμα 2

Βρίσκεται στα σχόλια

Πίνακας ρόλου καταχωρητών – Βήμα 2

\$v0	Carries the results and executes certain codes for printing and exiting the program via syscall
\$ra	Contains the return address and enables us to go back to right-after the 'jal myfactorial' by using jr(jump on register)

\$t0	Temporary register used in order to check whether we've reached a leaf
\$s4	Contains the result of the division of the factorial of 5 and the multiplication of the factorials of 2 and 3
\$sp	\$sp(stack pointer) is used to allocate and deallocate memory according to our needs in order to be able to have enough memory space to store and load words

Execute, Snapshot, MARS - Βήμα 2

The screenshot shows the MARS IDE with the following assembly code:

```

1: .text
2: .globl _start, 0
3: .globl _start, 0
4: .globl _start, 0
5: .globl _start, 0
6: .globl _start, 0
7: .globl _start, 0
8: .globl _start, 0
9: .globl _start, 0
10: .globl _start, 0
11: .globl _start, 0
12: .globl _start, 0
13: .globl _start, 0
14: .globl _start, 0
15: .globl _start, 0
16: .globl _start, 0
17: .globl _start, 0
18: .globl _start, 0
19: .globl _start, 0
20: .globl _start, 0
21: .globl _start, 0
22: .globl _start, 0
23: .globl _start, 0
24: .globl _start, 0
25: .globl _start, 0
26: .globl _start, 0
27: .globl _start, 0
28: .globl _start, 0
29: .globl _start, 0
30: .globl _start, 0
31: .globl _start, 0
32: .globl _start, 0
33: .globl _start, 0
34: .globl _start, 0
35: .globl _start, 0
36: .globl _start, 0
37: .globl _start, 0
38: .globl _start, 0
39: .globl _start, 0
40: .globl _start, 0
41: .globl _start, 0
42: .globl _start, 0
43: .globl _start, 0
44: .globl _start, 0
45: .globl _start, 0
46: .globl _start, 0
47: .globl _start, 0
48: .globl _start, 0
49: .globl _start, 0
50: .globl _start, 0
51: .globl _start, 0
52: .globl _start, 0
53: .globl _start, 0
54: .globl _start, 0
55: .globl _start, 0
56: .globl _start, 0
57: .globl _start, 0
58: .globl _start, 0
59: .globl _start, 0
60: .globl _start, 0
61: .globl _start, 0
62: .globl _start, 0
63: .globl _start, 0
64: .globl _start, 0
65: .globl _start, 0
66: .globl _start, 0
67: .globl _start, 0
68: .globl _start, 0
69: .globl _start, 0
70: .globl _start, 0
71: .globl _start, 0
72: .globl _start, 0
73: .globl _start, 0
74: .globl _start, 0
75: .globl _start, 0
76: .globl _start, 0
77: .globl _start, 0
78: .globl _start, 0
79: .globl _start, 0
80: .globl _start, 0
81: .globl _start, 0
82: .globl _start, 0
83: .globl _start, 0
84: .globl _start, 0
85: .globl _start, 0
86: .globl _start, 0
87: .globl _start, 0
88: .globl _start, 0
89: .globl _start, 0
90: .globl _start, 0
91: .globl _start, 0
92: .globl _start, 0
93: .globl _start, 0
94: .globl _start, 0
95: .globl _start, 0
96: .globl _start, 0
97: .globl _start, 0
98: .globl _start, 0
99: .globl _start, 0
100: .globl _start, 0

```

The registers window on the right shows the state of the registers, with \$t0 containing the value 120.

The screenshot shows the MARS IDE with the following assembly code:

```

1: .text
2: .globl _start, 0
3: .globl _start, 0
4: .globl _start, 0
5: .globl _start, 0
6: .globl _start, 0
7: .globl _start, 0
8: .globl _start, 0
9: .globl _start, 0
10: .globl _start, 0
11: .globl _start, 0
12: .globl _start, 0
13: .globl _start, 0
14: .globl _start, 0
15: .globl _start, 0
16: .globl _start, 0
17: .globl _start, 0
18: .globl _start, 0
19: .globl _start, 0
20: .globl _start, 0
21: .globl _start, 0
22: .globl _start, 0
23: .globl _start, 0
24: .globl _start, 0
25: .globl _start, 0
26: .globl _start, 0
27: .globl _start, 0
28: .globl _start, 0
29: .globl _start, 0
30: .globl _start, 0
31: .globl _start, 0
32: .globl _start, 0
33: .globl _start, 0
34: .globl _start, 0
35: .globl _start, 0
36: .globl _start, 0
37: .globl _start, 0
38: .globl _start, 0
39: .globl _start, 0
40: .globl _start, 0
41: .globl _start, 0
42: .globl _start, 0
43: .globl _start, 0
44: .globl _start, 0
45: .globl _start, 0
46: .globl _start, 0
47: .globl _start, 0
48: .globl _start, 0
49: .globl _start, 0
50: .globl _start, 0
51: .globl _start, 0
52: .globl _start, 0
53: .globl _start, 0
54: .globl _start, 0
55: .globl _start, 0
56: .globl _start, 0
57: .globl _start, 0
58: .globl _start, 0
59: .globl _start, 0
60: .globl _start, 0
61: .globl _start, 0
62: .globl _start, 0
63: .globl _start, 0
64: .globl _start, 0
65: .globl _start, 0
66: .globl _start, 0
67: .globl _start, 0
68: .globl _start, 0
69: .globl _start, 0
70: .globl _start, 0
71: .globl _start, 0
72: .globl _start, 0
73: .globl _start, 0
74: .globl _start, 0
75: .globl _start, 0
76: .globl _start, 0
77: .globl _start, 0
78: .globl _start, 0
79: .globl _start, 0
80: .globl _start, 0
81: .globl _start, 0
82: .globl _start, 0
83: .globl _start, 0
84: .globl _start, 0
85: .globl _start, 0
86: .globl _start, 0
87: .globl _start, 0
88: .globl _start, 0
89: .globl _start, 0
90: .globl _start, 0
91: .globl _start, 0
92: .globl _start, 0
93: .globl _start, 0
94: .globl _start, 0
95: .globl _start, 0
96: .globl _start, 0
97: .globl _start, 0
98: .globl _start, 0
99: .globl _start, 0
100: .globl _start, 0

```

The registers window on the right shows the state of the registers, with \$t0 containing the value 120.